Snake OS: A Real-Time Terminal System Simulation

IEEE Technical Report

Authors:
Abhijeet Kumar Shah, Ayush Dev, Aditya Raj Sharma

Abstract

Snake OS is a low-level terminal-based system simulation developed in C that demonstrates fundamental Operating System principles including process scheduling, custom memory management, and hardware-level driver abstraction. By implementing a monolithic kernel loop, a 64 KB virtual RAM (VRAM) subsystem with a custom heap allocator, and a bitwise mathematical engine, the project avoids the standard C library for core logic. The result is a high-performance, real-time interactive environment that simulates an embedded kernel's behavior through the medium of a classic arcade game.

I. Introduction

Traditional high-level programming often abstracts away the hardware interactions that define an Operating System. Snake OS was designed to peel back these layers by recreating essential OS components within a POSIX terminal environment. The project's primary objective was to build a "bare-metal" style application where every system call—from memory allocation to arithmetic operations—is handled by custom-built internal modules.

The system is structured as a **Monolithic Kernel** where the main event loop serves as the CPU scheduler, switching between different "Process States" (BOOT, RUNNING, EXCEPTION, SHUTDOWN) while managing device drivers for keyboard interrupts and ANSI-based video output.

II. System Architecture

A. Monolithic Kernel Loop

The kernel's core resides in `main.c`, which maintains a Finite State Machine (FSM). The state transitions are governed by user input and internal "interrupts" (collisions).

- **Start State:** Boots the system, initializes VRAM, and displays system metadata.
- **Running State:** The high-frequency execution loop where the scheduler triggers movement updates, collision detection, and screen flushing.
- **Game Over State:** Acts as a "Kernel Panic" or process exception handler, providing diagnostics on memory leaks and final system stats.

B. Virtual RAM (VRAM) & Allocator

One of the most critical components is `memory.c`. The system allocates a static **64 KB block** of memory to serve as the system heap.

- **Block Headers:** Each allocation includes an in-band header containing a `0xDEADBEEF` magic sentinel for integrity checks, the block size, and a free-flag.
- **Allocation Strategy:** The system uses a hybrid **Bump + Free-list** allocator. If the free-list cannot satisfy a request, the bump pointer (`vram_top`) expands the active region.
- **Alignment:** All allocations are 4-byte aligned using bitwise masking to ensure CPU-friendly memory access.

C. Bitwise Math Engine

To simulate hardware lacking a dedicated Floating Point Unit (FPU) or even standard multipliers, `math.c` implements all arithmetic using only bitwise shifts (`<<` , `>>`) and logical gates (`&` , `|` , `^` , `~`).

- **Multiplication:** Implemented via the **Russian Peasant (Shift-and-Add)** algorithm.
- **Division:** Uses a **Non-restoring Long Division** approach, processing one bit at a time.
- **PRNG:** A 32-bit **XOR-Shift generator** provides high-speed pseudo-random numbers for food spawning, ensuring no reliance on `<stdlib.h>`'s `rand()` .

III. System Software Stack

A critical design choice in Snake OS is the absolute avoidance of the standard C library (`libc`) for core runtime operations. This necessitates a layered utility stack:

- **Low-Level Math (ALU):** `math.c` provides the foundation, implementing signed multiplication and division using bit-shifting.
- **String Utilities:** `string.c` builds upon the ALU to provide `my_int_to_str` , allowing the kernel to display dynamic system statistics (like VRAM usage and scores) without using `printf` .
- **Game Engine:** `snake.c` consumes these utilities to manage the dynamic body of the snake, which is stored as a series of pointers in the simulated heap.

IV. Implementation Details

Module	Core Logic	OS Principle
`main.c`	FSM-based Event Loop	Process Scheduling & Lifecycle
`memory.c`	Free-list & Bump Allocator	Heap Management & Fragmentation
`math.c`	Shift-and-Add, Long Division	CPU ALU Simulation
`string.c`	Null-terminated byte string ops	Kernel Utility Library
`snake.c`	Memory-mapped Coordinate Update	Process Logic & Resource Guarding
`keyboard.c`	`termios` Raw Mode Polling	I/O Interrupt Handling
`screen.c`	ANSI Escape Sequences	Device Driver Abstraction

V. Results and Analysis

Testing of the system demonstrated a highly responsive interactive environment with minimal CPU overhead.

- **Memory Efficiency:** The 64 KB VRAM was found to be more than sufficient for the snake's growth, with the free-list effectively reusing memory from previous segments.
- **Arithmetic Precision:** The bitwise division and multiplication engines passed all sanity checks against standard C operators, confirming the robustness of the ALU simulation.
- **Flicker-Free I/O:** The software framebuffer approach achieved 60+ FPS equivalent performance in the terminal, demonstrating efficient "hardware" communication.

VI. Future Scope

The modular architecture of Snake OS allows for several advanced extensions that would further simulate modern OS complexities:

- **Process Concurrency:** Integrating a pre-emptive scheduler to manage multiple concurrent tasks, such as autonomous AI entities or background system monitors.
- **Virtual File System (VFS):** Implementing a basic inode-based filesystem within a dedicated VRAM segment to persist system logs and persistent high scores.
- **Memory Paging:** Transitioning from a flat memory model to a simulated paged architecture with virtual-to-physical address translation.
- **Network Stack Simulation:** Developing a socket-based driver to allow "Inter-Kernel Communication" between two separate instances of the system.
- **AI Kernel Modules:** Integrating neural network logic implemented via bitwise math for autonomous snake navigation.

VII. Conclusion

Snake OS successfully demonstrates that complex system behaviors can be simulated using low-level C programming. By removing dependencies on the standard library and implementing core OS primitives manually, the project provides a deep understanding of how a kernel manages resources, handles I/O, and executes logic at the lowest level of the software stack.

References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*.
2. Marsaglia, G. (2003). *Xorshift RNGs*. Journal of Statistical Software.
3. IEEE Std 1003.1 (POSIX) Terminal Interface Specifications.